

Introduction to SDLC & Core Methodologies

Imagine trying to build a house without a blueprint. You might start with the kitchen, then realize you forgot the bathroom, and end up with a roof that doesn't fit. Building software is the same. To avoid this chaos, development teams follow a Software Development Life Cycle, or SDLC. This is a structured process that guides a project from its initial idea to its final release and beyond. It provides a framework for delivering high-quality software that **meets user expectations**, on time and within budget.

While different projects may follow different paths, most SDLCs include several key stages.

Requirements Gathering and Analysis: *The "What" Phase*

This is the foundation of the entire project. The team meets with **stakeholders** (clients, users, managers) to define exactly what the software needs to do. What problems will it solve? Who will use it? The **output** of this stage is a detailed document called a Software Requirements Specification (SRS) or a **Product Backlog** (in Agile). A clear requirement here **prevents costly misunderstandings** later.

Design: *The "How" Phase*

In this phase, system architects and senior developers take the requirements and create the **technical blueprint**. They define the overall system architecture, choose the technology stack (like programming languages and databases), and design the user interface. Think of it as creating the detailed plans an architect gives to a construction crew. A solid design makes the next phase much smoother.

Implementation (or Coding): *The "Coding" Phase*

This is where the actual work of writing code happens. Developers **follow the design specifications** to build the software's features. They turn the design documents into a functional product. This stage is often the longest, but it relies entirely on the quality of the previous stages.

Testing: *The "Validation" Phase*

Once the code is written, it must be verified. A **Quality Assurance (QA) team**, or the developers themselves, rigorously test the software to find bugs (errors) and ensure it meets the requirements defined in the first stage. This can involve everything from testing individual units of code (unit testing) to testing the entire system together (integration testing). The goal is to identify and fix issues before the user ever sees them.

Deployment: *The "Release" Phase*

After the software has been tested and approved, it's time to release it to the users. This is the deployment phase. The software is installed on servers, made available in an app store, or distributed to users. For some projects, this is a big event. For modern web applications, deployment can happen many times a day.

Maintenance: *The "Live" Phase*

The launch is not the end. After the software is in use, it enters the **maintenance** phase. Users will inevitably find new bugs, request new features, or the environment (like an operating system) might change. The development team must **provide ongoing support**, fixing issues

and **releasing patches** and updates to keep the software running smoothly. This is often the longest phase of the entire life cycle.

Discussion:

1. Is a detailed blueprint always necessary, or are there times when it is better to just "start building" and figure things out as you go? Why?
2. In your opinion, which single stage of the SDLC is the most important for a project's success? Which stage, if done poorly, would cause the biggest failure? Justify your choice.
3. Imagine the stakeholders (clients, managers) say they want "a fast, modern, and user-friendly website." Is this a good requirement? Why not? What problems could this vague language cause during the Implementation phase?
4. Why do you think many companies and project managers underestimate the cost and effort of the Maintenance phase? Is a project ever truly "finished"?
5. Think about the Testing phase. Which do you think is more expensive to fix: a bug found during the Testing phase, or the same bug found by a user after Deployment? Why? What costs are involved beyond just fixing the code?
6. Should developers simply "follow the specifications," or should they have the freedom to be creative and change the design if they see a better way to build something? How much creativity should a developer have?
7. Is software a success if it is delivered on time and within budget, but users hate it and refuse to use it? How would you redefine what a "successful" software project looks like?

1. Match the terms with their definitions.

Term	Definition
1. SDLC	a) A detailed document created during the requirements phase that outlines all the functions the software must perform.
2. Stakeholders	b) The phase where a Quality Assurance team rigorously tests the software to find errors.
3. Software Requirements Specification (SRS)	c) An error, flaw, or fault in a computer program that causes it to produce an incorrect result.
4. Technology Stack	d) The clients, users, or managers who have an interest in the project's outcome.
5. Testing	e) The combination of programming languages, frameworks, and databases chosen to build the application.

6. Bug	f) A structured process that guides a project from its initial idea through development and maintenance.
--------	--

7. Maintenance	g) The ongoing phase where the team provides support, fixes new issues, and releases updates after the software is live.
----------------	--

2. Read each description carefully. Write down the order and name of the stage (e.g., Design, Testing, etc.).

- A. Programmers finally get involved, writing the actual code to turn the specifications into a functional product.
- B. System architects create the technical blueprint. They define the system architecture and choose the technology stack.
- C. The foundation of the project. The team meets with stakeholders to define exactly what problems need to be solved and what features are required.
- D. The launch is not the end. The team provides ongoing support, fixing new bugs and releasing patches to keep the software running smoothly.
- E. A Quality Assurance team rigorously checks for bugs. They test everything from small code units to the entire system to ensure quality.
- F. After passing all checks, the software is finally released to users. It is installed on servers or published in an app store.

3. Read the sentences below and fill in the gaps with the correct words from the box. Use the words in bold from the text to help you.

Word Bank

stakeholders

output

follow the specifications

maintenance

deployment

*prevent costly
misunderstandings*

provide ongoing support

release patches

1. At the beginning of a project, the team meets with all _____ (clients, users, managers) to understand their needs.

2. The _____ of the requirements phase is a detailed document called a Software Requirements Specification (SRS).
3. Getting clear requirements at the start helps _____ later in the project.
4. During the coding phase, developers must carefully _____ created by the system architects.
5. After the software has been tested and approved, it moves into the _____ phase and is released to users.
6. Once the software is live, the team enters the _____ phase to handle any issues that appear.
7. When users report new problems, the development team must _____ to fix them.
8. Even after launch, the team continues to _____ by answering questions and fixing unexpected errors.

The Core Roles in an IT Project

Building software requires a diverse group of people with different skills. Understanding who does what on a development team is essential for anyone working in or with the tech industry. Every successful project relies on a mix of technical and non-technical roles that must work together effectively.

At the business end of the project sits the **Product Owner (PO)**, a business-focused role that represents the **stakeholders**—the clients, users, and managers who have an interest in the project. The Product Owner is responsible for defining the vision by translating business goals into clear, actionable tasks. They own and prioritize the **Product Backlog**, a living document listing every feature, change, or fix that might be needed. By setting priorities, they decide what the team works on next based on business value. When work is completed, they review it and decide if it meets the **Definition of Done**. The Product Owner works most closely with stakeholders, the Scrum Master, and the Development Team.

Supporting the process is the **Scrum Master (SM)**, who acts as a facilitator and coach. Unlike a traditional project manager, the Scrum Master is not a boss. Instead, they ensure the team follows Agile principles and works effectively. Their responsibilities include facilitating key events like **sprint planning**, the **daily stand-up**, and the **sprint retrospective**. When the team faces an obstacle that slows them down, the Scrum Master steps in to remove that **roadblock**. They also coach the team on Agile practices, helping them improve over time, and protect the team from outside distractions and interruptions. The Scrum Master works primarily with the Product Owner and the entire Development Team.

The heart of any project is the **Development Team** - the group of people who actually design, code, and test the product. In Agile environments, this team is **cross-functional** (including all necessary skills) and **self-organizing** (deciding internally how to work). During sprint planning, they assess the effort required for tasks. Their main job is writing code, turning requirements into a functional, working product. They also handle technical design decisions and take ownership of quality by writing **unit tests** and reviewing each other's code. Within the **Development Team**, there are several common specializations. **Front-end developers** build what the user sees and interacts with (the UI). **Back-end developers** work on server-side logic, databases, and APIs. **Full-stack developers** have skills in both front-end and back-end development. DevOps engineers focus on infrastructure, automation, and ensuring reliable **deployment**. The Development Team works most closely with the Product Owner and the QA Engineer.

Ensuring quality is the responsibility of the **Quality Assurance (QA) Engineer** (also called a Tester). QA engineers are responsible for making sure the software works correctly and meets requirements before it reaches users. Their work involves **test planning**, where they create detailed test cases to verify features work as expected. They perform **manual testing**, actively using the software to find bugs (errors or crashes). For larger projects, they also write code for **automated testing**, checking that new code hasn't broken existing functionality (a practice called **regression testing**). When they find issues, they handle **bug reporting**,

documenting problems clearly, and then **verify** fixes once developers resolve them. QA engineers work most closely with Developers and the Product Owner.

Shaping how the product looks and feels is the **UI/UX Designer**. This role often splits into two specializations. UI (User Interface) Designers focus on visual presentation—the colors, typography, buttons, and icons. UX (User Experience) Designers focus on usability and how the product feels to use. UX designers conduct **user research** through interviews and tests to understand user needs. They create **user flows**, mapping the steps users take to complete tasks, and **develop wireframes**—basic sketches of screen layouts. UI designers handle the final **visual design** and build **design systems**, creating reusable components for consistent development. Together, they ensure the software is not only functional but also intuitive and enjoyable. UI/UX designers work most closely with the Product Owner and Front-end Developers.

Finally, in organizations using **Waterfall** or hybrid approaches, there is often a **Project Manager (PM)**. Unlike the Scrum Master, who serves the team and protects the process, the Project Manager serves the business and manages the plan. The PM focuses on the "iron triangle" of project management: **scope, time, and budget**. Their responsibilities include creating detailed project plans and schedules, tracking costs and ensuring the project stays within budget, reporting progress to stakeholders, and allocating resources.

	Definition	Your Answer (Find the bolded term)
1	The client, users, and managers who have an interest in the project.	_____
2	A living document listing every feature, change, or fix that might be needed.	_____
3	A shared understanding of when work is complete (coded, tested, approved).	_____
4	A facilitator and coach who ensures the team follows Agile principles.	_____
5	A short, daily meeting where team members discuss progress.	_____
6	An obstacle that prevents the team from making progress.	_____

7	A team that includes all necessary skills (design, coding, testing).	_____
8	A team that decides internally how to do its work.	_____
9	Testing individual components of code to ensure they work correctly.	_____
10	Developers who build what the user sees and interacts with.	_____
11	Engineers who focus on infrastructure, automation, and reliable deployment.	_____
12	The process of making software available to users.	_____
13	A set of steps used to verify that a feature works correctly.	_____
14	An error or flaw in a computer program.	_____
15	Testing to ensure new code hasn't broken old features.	_____
16	To confirm that a bug fix actually solved the problem.	_____
17	Designers who focus on visual presentation—colors, typography, buttons.	_____
18	Designers who focus on usability and how the product feels.	_____
19	A basic sketch showing the layout of a screen.	_____
20	A collection of reusable components for consistent design.	_____
21	A linear, sequential development methodology.	_____
22	All the work that needs to be done in a project.	_____

Text discussion:

1. What is the Product Backlog, and who manages it?
2. What does it mean when a Product Owner "accepts" work?
3. Name three specific events (meetings) that the Scrum Master facilitates.
4. What does a QA Engineer do after a developer fixes a bug?
5. What is a "roadblock," and what does the Scrum Master do about it?
6. Which of the roles described sounds most interesting to you? Why?

Project Work: Plan a Software Project

Project Objectives

- Apply the six stages of the SDLC to a realistic project scenario.
- Define and assign key IT project roles (Product Owner, Scrum Master, Developer, QA, Designer).
- Use terminology from the lessons in written and spoken form.
- Collaborate effectively in a cross-functional team.
- Present a project plan clearly and professionally.

Choose one of the following project ideas:

Option	Project Idea	Client Request
A	Campus Connect App	A university wants a mobile app for students to find study groups, share notes, and get campus news.
B	MediTrack System	A small clinic needs software to manage patient appointments, medical records, and billing.
C	EcoShop Platform	An eco-friendly startup wants an e-commerce website to sell sustainable products.
D	TeamFlow Tool	A company needs a simple internal tool for task management and team collaboration.

Client Requirements (for all projects):

- The solution must be user-friendly and reliable.
- The client has a limited budget and wants the most valuable features first.
- The client expects a clear plan before development starts.

Presentation Structure:

1. Introduction: What is the project? Who is on the team?
2. Requirements: What are the top features? Show your Product Backlog.
3. Design: Show your wireframes and explain the user experience.
4. Implementation & Testing: How will you build it? How will you test it?
5. Deployment & Maintenance: How will you release it and support it?
6. Conclusion: Why should the client choose your team?

